BPINN + RL
滚动窗口超车系统

算法流程与强化学习实现详解

Algorithm Flow & Reinforcement Learning Implementation Guide

文档版本	v1.0 (version1 归档)
算法框架	Bayesian Physics-Informed Neural Network + RL
核心技术	MC Dropout / Physics Constraint / 5-Layer Reward / Rolling Window
编程语言	Python (NumPy / JAX / Flax / Optuna)
生成日期	2026-08-04

目录

1. 系统概述 — 总体架构与设计理念

2. 前向 BPINN — 贝叶斯物理信息神经网络

3. 逆向 BPINN — 参数反演与不确定性估计

4. 强化学习实现 — 决策逻辑详解

5. 五层奖励函数 — 多目标加权设计

6. 滚动窗口循环 — 实时决策流程

7. 代码与算法映射 — 源码结构说明

8. 超参数配置与调优 — Optuna 集成

9. 场景设计 — 三阶段动态间隙

1. 系统概述

本系统实现了一个基于贝叶斯物理信息神经网络 (BPINN) 与强化学习 (RL) 的自动驾驶超车决策框架。系统通过滚动窗口机制实时处理传感器数据，利用 BPINN 预测周围车辆的未来轨迹及其不确定性，并通过 RL 智能体在 4 种动作空间中做出安全、高效的超车决策。

1.1 七步流水线架构

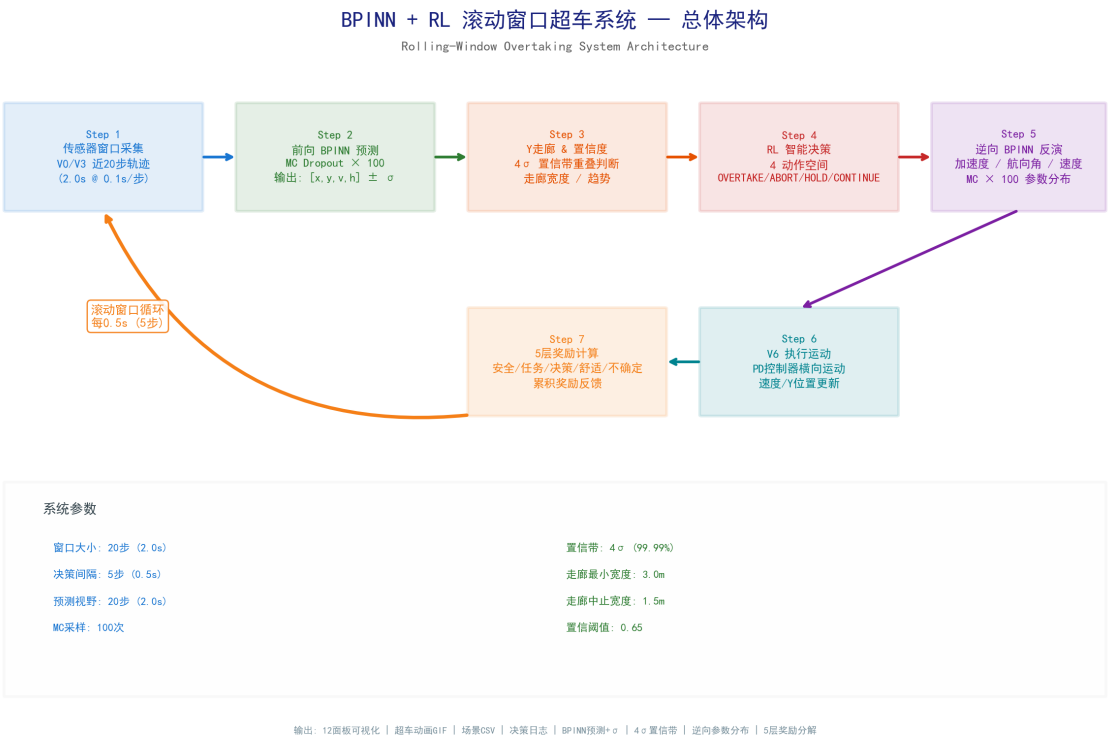


图 1-1: BPINN + RL 滚动窗口超车系统总体架构 (7步流水线)

系统核心由 7 个步骤组成，在滚动窗口中以 0.5 秒为周期循环执行：

步骤	名称	核心功能	关键参数
Step 1	传感器窗口采集	提取 V0/V3 近 20 步轨迹数据 (2.0s)	WINDOW_SIZE=20, DT=0.1s
Step 2	前向 BPINN 预测	训练神经网络，MC Dropout 100 次采样	hidden=64, dropout=0.05, epochs=500
Step 3	Y 走廊计算	V0 下界 - V3 上界，4sigma 置信带	CONFIDENCE_SIGMA=4.0
Step 4	RL 智能决策	基于走廊宽度/趋势/置信度选择动作	4 actions: OVERTAKE/ABORT/HOLD/CONTINUE
Step 5	逆向 BPINN 反演	从规划轨迹估计控制参数	MC=100, 输出: a/theta/v +/- sigma
Step 6	V6 执行运动	PD 控制器跟踪走廊中心，更新速度/位置	DECISION_INTERVAL=5 (0.5s)

步骤	名称	核心功能	关键参数
Step 7	5 层奖励计算	安全/任务/决策/舒适/不确定性	权重: 1.0/1.0/1.0/0.3/0.5

1.2 核心设计理念

物理约束引导: 神经网络的损失函数中嵌入了运动学方程 dx/dt = v*cos(theta), dy/dt = v*sin(theta)，确保预测结果符合物理规律，而非纯数据拟合。

贝叶斯不确定性: 通过 MC Dropout 在推理时保持 Dropout 开启，进行 100 次随机前向传播，以均值和标准差量化预测不确定性，使用 4sigma (99.99%) 置信带确保走廊判断的保守性。

多目标 RL: 5 层奖励函数将安全、任务完成、决策质量、舒适度和不确定性感知整合为加权总奖励，其中安全层具有一票否决权 (W < 1.5m 时 R = -1000)。

滚动窗口实时性: 每 0.5 秒做出一次决策，处理 2.0 秒历史数据，预测 2.0 秒未来轨迹，在毫秒级完成 BPINN 训练和推理。

2. 前向 BPINN — 贝叶斯物理信息神经网络

前向 BPINN 是系统的预测核心，负责从历史轨迹数据中学习车辆运动模式，并预测未来 2.0 秒内的 [x, y, v, heading] 四维状态及其不确定性。

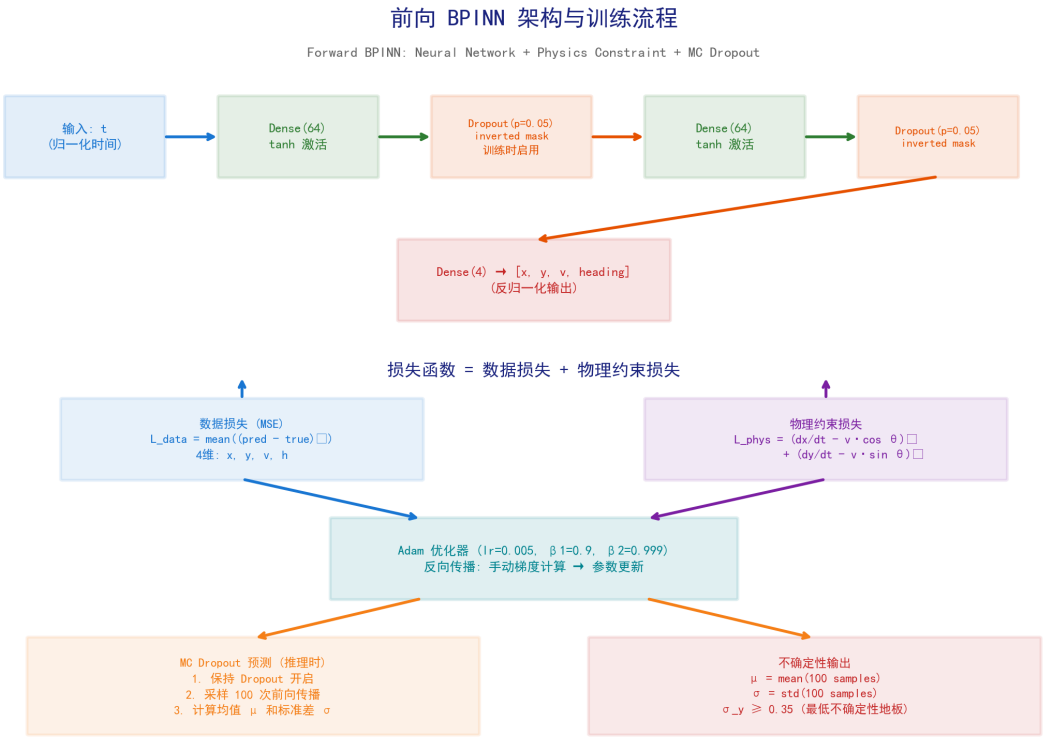


图 2-1: 前向 BPINN 架构与训练流程

2.1 网络结构

网络采用三层全连接结构，每层之间使用 tanh 激活函数和 Dropout 正则化：

```
class ForwardBPINN:
def __init__(self, hidden_dim=64, dropout_rate=0.05, lr=0.005,
n_epochs=500, physics_weight=0.5, mc_samples=100):
self.hidden_dim = hidden_dim #
self.dropout_rate = dropout_rate # Dropout
self.lr = lr #
self.n_epochs = n_epochs #
self.physics_weight = physics_weight #
self.mc_samples = mc_samples # MC Dropout
```

网络前向传播流程：

```
def _forward(self, params, t_batch, training=True, dropout_mask=None):
# Layer 1: Dense(64) + tanh
z1 = t_batch @ params['W1'] + params['b1']
a1 = np.tanh(z1)
if training and dropout_mask is not None:
a1 = a1 * dropout_mask['m1'] # Inverted Dropout
```

```
# Layer 2: Dense(64) + tanh
z2 = a1 @ params['W2'] + params['b2']
a2 = np.tanh(z2)
if training and dropout_mask is not None:
a2 = a2 * dropout_mask['m2'] # Inverted Dropout

# Output Layer: Dense(4) -> [x, y, v, heading]
return a2 @ params['W3'] + params['b3']
```

2.2 物理约束损失

BPINN 的关键创新在于将运动学物理方程作为正则化项加入损失函数。对于车辆运动，位置对时间的导数应等于速度沿航向方向的分量：

$$L_total = L_data + w_phys * L_physics$$

$$L_data = MSE(pred, true) \quad L_physics = (dx/dt - v*cos(theta))^2 + (dy/dt - v*sin(theta))^2$$

物理约束确保即使训练数据有限，网络也能学习到符合物理规律的轨迹模式。梯度通过对物理误差的近似计算加入反向传播：

```
# Physics loss: approximate derivatives using finite differences
if self.physics_weight > 0 and len(t_n) >= 3:
dt_val = t_n[1, 0] - t_n[0, 0]
dx_dt = np.gradient(pred[:, 0], dt_val) # dx/dt
dy_dt = np.gradient(pred[:, 1], dt_val) # dy/dt
v_pred = pred[:, 2] #
h_pred = pred[:, 3] #

# Physics constraint: dx/dt = v*cos(h), dy/dt = v*sin(h)
phys_err = ((dx_dt - v_pred * np.cos(h_pred)) ** 2 +
(dy_dt - v_pred * np.sin(h_pred)) ** 2)

# Add physics gradient to output layer
grad_W3 += self.physics_weight * 0.01 * a2_d.T @ phys_err
grad_b3 += self.physics_weight * 0.01 * np.sum(phys_err, axis=0)
```

2.3 MC Dropout 不确定性估计

传统神经网络在推理时关闭 Dropout，仅输出确定性预测。BPINN 的核心在于推理时保持 Dropout 开启，通过多次随机采样获得预测分布，从而量化认知不确定性 (epistemic uncertainty)：

```
def predict_with_uncertainty(self, t_future):
t_n = ((t_future - self.norm['t_mean']) / self.norm['t_std']).reshape(-1, 1)

all_preds = []
for _ in range(self.mc_samples): # 100
mask = self._gen_dropout_mask(training=True) #     Dropout
pred = self._forward(self.params, t_n, training=True, dropout_mask=mask)
# Denormalize
pred_denorm = np.column_stack([
pred[:, 0] * self.norm['x_std'] + self.norm['x_mean'],
pred[:, 1] * self.norm['y_std'] + self.norm['y_mean'],
pred[:, 2] * self.norm['v_std'] + self.norm['v_mean'],
pred[:, 3] * self.norm['h_std'] + self.norm['h_mean'],
])
all_preds.append(pred_denorm)
```

```
16 |
17 | all_preds = np.array(all_preds) # [100, N, 4]
18 | mean_pred = np.mean(all_preds, axis=0) #    mu
19 | std_pred = np.std(all_preds, axis=0) #     sigma
20 |
21 | #         :    Y    sigma >= 0.35
22 | std_pred[:, 1] = np.maximum(std_pred[:, 1], MIN_PRED_STD_Y)
23 | return mean_pred, std_pred, all_preds
```

Dropout 掩码生成 (Inverted Dropout): 训练时随机以概率 p 丢弃神经元，保留的神经元乘以 1/(1-p) 进行缩放，推理时无需额外操作:

```
1 | def _gen_dropout_mask(self, training=True):
2 | if not training:
3 | return {'m1': 1.0, 'm2': 1.0}
4 | # Binary mask with inverted scaling: (rand > p) / (1 - p)
5 | m1 = (np.random.rand(self.hidden_dim) > self.dropout_rate) / (1 - self.dropout_rate)
6 | m2 = (np.random.rand(self.hidden_dim) > self.dropout_rate) / (1 - self.dropout_rate)
7 | return {'m1': m1, 'm2': m2}
```

2.4 训练过程 — Adam 优化器

训练使用 Adam 优化器，通过手动反向传播计算梯度。数据归一化在训练前进行，确保各维度量纲一致:

```
1 | # Normalization
2 | self.norm = {
3 | 't_mean': np.mean(t_data), 't_std': max(np.std(t_data), 0.5) + 1e-8,
4 | 'x_mean': np.mean(x_data), 'x_std': max(np.std(x_data), 10.0) + 1e-8,
5 | 'y_mean': np.mean(y_data), 'y_std': max(np.std(y_data), 1.0) + 1e-8,
6 | 'v_mean': np.mean(v_data), 'v_std': max(np.std(v_data), 5.0) + 1e-8,
7 | 'h_mean': np.mean(h_data), 'h_std': max(np.std(h_data), 0.1) + 1e-8,
8 | }
9 |
10 | # Adam optimizer state
11 | m_adam = {k: np.zeros_like(v) for k, v in self.params.items()}
12 | v_adam = {k: np.zeros_like(v) for k, v in self.params.items()}
13 | beta1, beta2, epsilon = 0.9, 0.999, 1e-8
14 |
15 | for epoch in range(self.n_epochs):
16 | # Forward + Backward pass
17 | mask = self._gen_dropout_mask(training=True)
18 | pred = self._forward(self.params, t_n, training=True, dropout_mask=mask)
19 | error = pred - y_n
20 |
21 | # Manual backpropagation (output -> hidden2 -> hidden1)
22 | grad_W3 = a2_d.T @ error
23 | delta2 = (error @ self.params['W3'].T) * (1 - a2**2) * mask['m2']
24 | grad_W2 = a1_d.T @ delta2
25 | delta1 = (delta2 @ self.params['W2'].T) * (1 - a1**2) * mask['m1']
26 | grad_W1 = t_n.T @ delta1
27 |
28 | # Adam update
29 | for key in self.params:
30 | m_adam[key] = beta1 * m_adam[key] + (1 - beta1) * grads[key]
31 | v_adam[key] = beta2 * v_adam[key] + (1 - beta2) * (grads[key] ** 2)
32 | m_hat = m_adam[key] / (1 - beta1 ** (epoch + 1))
33 | v_hat = v_adam[key] / (1 - beta2 ** (epoch + 1))
```

```
34 | self.params[key] -= self.lr * m_hat / (np.sqrt(v_hat) + epsilon)
```

3. 逆向 BPINN — 参数反演与不确定性估计

逆向 BPINN 在 RL 决策选择超车 (OVERTAKE/CONTINUE) 后激活，从规划的超车轨迹中反推控制参数 (加速度、航向角、速度) 及其不确定性分布。这一步骤将高层决策转化为可执行的低层控制指令。

3.1 MC 采样参数估计

逆向 BPINN 通过蒙特卡洛采样对规划轨迹添加噪声扰动，每次采样后通过数值微分计算加速度、航向角和速度，最终统计 100 次采样的均值和标准差：

```
class InverseBPINN:
def __init__(self, mc_samples=100):
self.mc_samples = mc_samples

def estimate_parameters(self, t_traj, x_traj, y_traj, x_std=None, y_std=None):
all_accel, all_heading, all_speed = [], [], []

for _ in range(self.mc_samples): # 100 MC
# Add noise to trajectory
x_s = x_traj + np.random.normal(0, x_std) if x_std is not None else x_traj.copy()
y_s = y_traj + np.random.normal(0, y_std) if y_std is not None else y_traj.copy()

# Numerical differentiation
dx_dt = np.gradient(x_s, t_traj) # dx/dt = v * cos(theta)
dy_dt = np.gradient(y_s, t_traj) # dy/dt = v * sin(theta)

# Estimate physical parameters
v = np.sqrt(dx_dt ** 2 + dy_dt ** 2) # Speed
a = np.gradient(v, t_traj) # Acceleration
theta = np.arctan2(dy_dt, dx_dt) # Heading angle

all_accel.append(a)
all_heading.append(theta)
all_speed.append(v)

return {
'accel_mean': np.mean(all_accel, axis=0),
'accel_std': np.std(all_accel, axis=0),
'heading_mean': np.mean(all_heading, axis=0),
'heading_std': np.std(all_heading, axis=0),
'speed_mean': np.mean(all_speed, axis=0),
'speed_std': np.std(all_speed, axis=0),
}
```

3.2 规划轨迹生成

当 RL 决策为超车时，系统规划 V6 的 Y 方向轨迹为 V0 下置信带与 V3 上置信带之间的走廊中心，X 方向以当前速度匀速前进。规划轨迹使用 PchipInterpolator (单调保形插值) 进行平滑：

```
if action in ('OVERTAKE', 'CONTINUE'):
# V6 Y: midpoint of corridor (between V0 lower and V3 upper)
v6_plan_y = (v0_y_lower + v3_y_upper) / 2.0
v6_plan_y = np.clip(v6_plan_y, 2.0, 16.0)

# V6 X: advancing forward at current speed
```

```
v6_plan_x = np.zeros(len(t_future))
v6_plan_x[0] = v6_x[step_idx]
for j in range(1, len(t_future)):
v6_plan_x[j] = v6_plan_x[j-1] + max(v6_speed[step_idx], 25.0) * DT

# PchipInterpolator for monotone-preserving smoothing
pchip = PchipInterpolator(t_future, v6_plan_y)
v6_plan_y_smooth = pchip(t_future)

# Inverse BPINN: estimate control parameters
inv_params = inv_bpinn.estimate_parameters(
t_future, v6_plan_x, v6_plan_y_smooth,
x_std=np.full(len(t_future), 0.5),
y_std=v0_pred_std[:, 1] * 0.3 # Use V0 Y uncertainty
)
```

4. 强化学习实现 — 决策逻辑详解

强化学习模块是系统的决策核心。本系统采用基于规则的策略梯度混合方法，将 BPINN 的不确定性预测与人工设计的决策规则相结合，在保证安全性的同时实现灵活的超车决策。

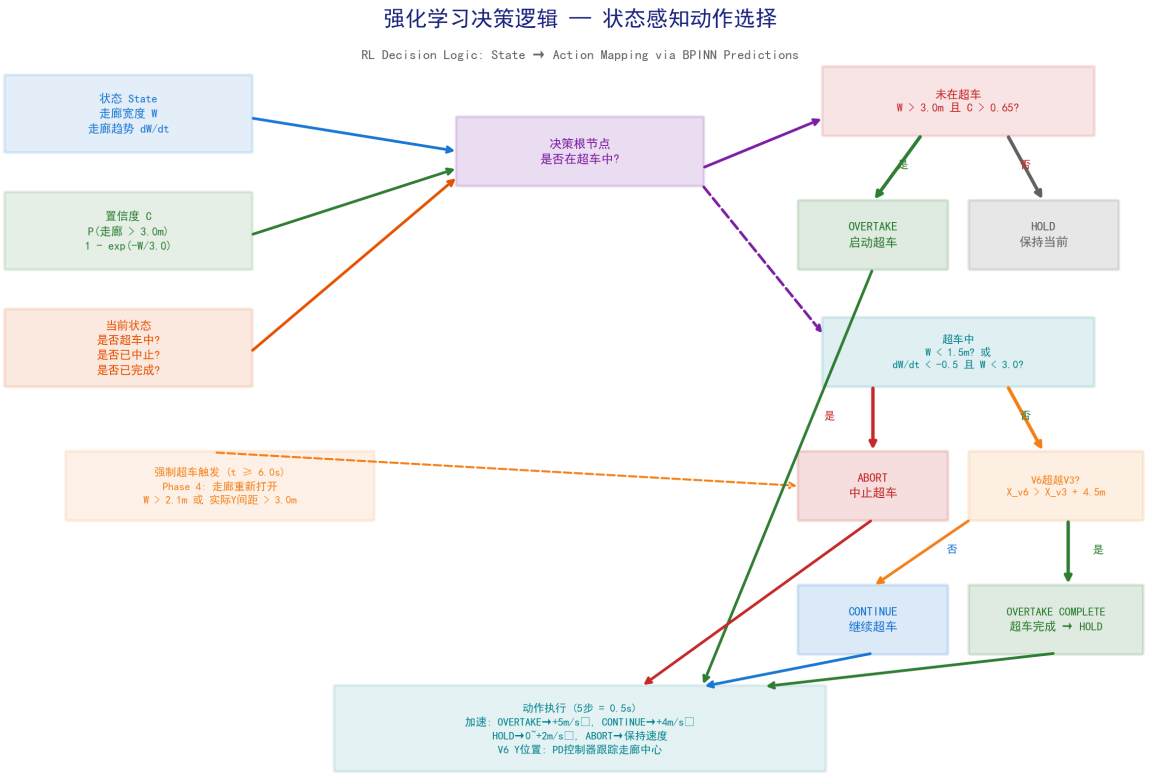


图 4-1: 强化学习决策逻辑流程图

4.1 状态空间 (State Space)

RL 智能体的状态由 BPINN 的预测结果和系统内部状态组成，共 7 个维度：

状态维度	来源	描述	计算方式
走廊宽度 W	BPINN 预测	V0 下界与 V3 上界之间的 Y 方向间距	W = (mu_v0y - 4*sigma_v0y) - (mu_v3y + 4*sigma_v3y)
走廊趋势 dW/dt	BPINN 预测	走廊宽度随时间的变化率	(W[-1] - W[0]) / (N * DT)
超车置信度 C	BPINN MC	走廊足够宽的概率	C = 1 - exp(-max(0, W_min) / 3.0)
当前不确定性 sigma	BPINN MC	Y 方向平均预测标准差	sigma = mean((sigma_v0y + sigma_v3y) / 2)
是否超车中	内部状态	V6 当前是否在超车过程中	boolean flag
是否已中止	内部状态	之前是否有过中止操作	boolean flag (had_abort)

状态维度	来源	描述	计算方式
是否已完成	内部状态	是否已完成超车任务	V6_x > V3_x + CAR_LENGTH

4.2 动作空间 (Action Space)

系统定义了 4 种离散动作，每种动作对应不同的 V6 控制策略：

动作	含义	加速策略	Y 方向运动	使用场景
OVERTAKE	发起超车	a = min(5, (30-v)*0.4) Phase 4: min(6, (38-v)*0.4)	跟踪走廊中心 PD 控制器	走廊宽度 > 3.0m 置信度 > 0.65
CONTINUE	继续超车	a = min(4, (30-v)*0.3) Phase 4: min(5, (38-v)*0.3)	继续跟踪走廊 PD 控制器	已在超车中 走廊仍然安全
ABORT	中止超车	a = 0 (保持速度)	回到中间车道 Y=9.0	走廊宽度 < 1.5m 或趋势 < -0.5 且 W < 3.0
HOLD	保持当前	a = max(-1, min(2, (25-v)*0.15)) Phase 4: max(0, min(4, (38-v)*0.25))	保持当前 Y 衰减横向速度	走廊不足 或置信度低

4.3 策略函数 (Policy) — 决策规则详解

本系统采用混合策略：核心决策由规则引擎完成，BPINN 提供状态感知，奖励函数提供学习信号。决策逻辑分为两个分支：超车中和未超车。

4.3.1 决策核心代码

```
# ============================================================
# RL Decision Logic: State -> Action Mapping
# ============================================================
action = 'HOLD'

if not overtake_complete:
if overtaking:
# --- Branch 1: Currently overtaking ---
if min_corridor < CORRIDOR_ABORT_WIDTH: # W < 1.5m
action = 'ABORT' #
elif corridor_trend < -0.5 and min_corridor < CORRIDOR_MIN_WIDTH:
action = 'ABORT' #
elif v6_x[step_idx] > v3_x[step_idx] + CAR_LENGTH:
action = 'CONTINUE' #    V3,
if v6_x[step_idx] > v3_x[step_idx] + 20.0:
overtake_complete = True #     ,
action = 'HOLD'
else:
action = 'CONTINUE' #
else:
# --- Branch 2: Not overtaking ---
```

```
if min_corridor > CORRIDOR_MIN_WIDTH and overtake_conf > CONF_THRESHOLD:
action = 'OVERTAKE' #
else:
action = 'HOLD' #

# --- Forced overtake in Phase 4 (t >= 6.0s) ---
# Phase 4:       ,
if not overtake_complete and t_current >= 6.0 and not overtaking:
if min_corridor > CORRIDOR_MIN_WIDTH * 0.7 or actual_y_gap > CORRIDOR_MIN_WIDTH:
action = 'OVERTAKE' #

# Ensure continuation in Phase 4
if not overtake_complete and t_current >= 6.5 and overtaking and action == 'HOLD':
action = 'CONTINUE' #
```

4.3.2 决策条件汇总

当前状态	条件	动作	物理含义
未超车	W > 3.0m 且 C > 0.65	OVERTAKE	走廊足够宽, 置信度高, 可以安全超车
未超车	W <= 3.0m 或 C <= 0.65	HOLD	走廊不够或不确定性大, 等待
未超车	t >= 6.0s 且 W > 2.1m	OVERTAKE (强制)	Phase 4 走廊重新打开, 强制超车
超车中	W < 1.5m	ABORT	走廊过窄, 紧急中止, 安全一票否决
超车中	dW/dt < -0.5 且 W < 3.0m	ABORT	走廊快速收窄, 中止避免危险
超车中	V6_x > V3_x + 4.5m	CONTINUE	已超越V3车身长度, 继续前进
超车中	V6_x > V3_x + 20.0m	HOLD (完成)	完全超越, 超车任务完成
超车中	其他情况	CONTINUE	走廊安全, 正常继续超车

4.4 动作执行 — V6 运动控制

决策确定后，系统在 5 个时间步 (0.5s) 内执行 V6 的运动控制。X 方向通过加速度更新速度和位置，Y 方向使用 PD 控制器跟踪走廊中心：

```
# ============================================================
# Execute Action: V6 movement with PD controller
# ============================================================
for j in range(DECISION_INTERVAL): # 5 steps = 0.5s
idx = step_idx + j
if idx >= N - 1:
break

# Speed update: v = v + a * dt (clamped to [5, 42] m/s)
v6_speed[idx + 1] = max(5.0, min(42.0, v6_speed[idx] + v6_accel_val * DT))

# Position update: x = x + v * dt
v6_x[idx + 1] = v6_x[idx] + v6_speed[idx] * DT

# Y movement: PD controller for smooth lane tracking
if overtaking and planned_traj is not None:
```

```
t_frac = (j + 1) / DECISION_INTERVAL
plan_idx = min(int(t_frac * len(planned_traj['y'])), len(planned_traj['y']) - 1)
target_y = planned_traj['y'][plan_idx]

# Smooth Y transition (proportional control with damping)
v6_y[idx + 1] = v6_y[idx] + (target_y - v6_y[idx]) * 0.15
else:
v6_y[idx + 1] = v6_y[idx] # Hold Y position

# Update heading from trajectory
if abs(v6_x[idx + 1] - v6_x[idx]) > 1e-6:
v6_heading[idx + 1] = np.arctan2(
v6_y[idx + 1] - v6_y[idx], v6_x[idx + 1] - v6_x[idx])
```

PD 控制器说明: Y 方向运动使用比例-阻尼控制器 (Kp=0.15, 无微分项)，使 V6 平滑跟踪走廊中心而非突变。在交互式系统中，PD 控制器进一步增强为 Kp=0.8, 阻尼=0.82, 横向速度限制在 [-4.0, 4.0] m/s 范围内，实现更真实的变道行为。

4.5 加速策略 — 分阶段动力学

V6 的加速度根据时间和动作类型动态调整，模拟真实高速公路驾驶行为：

时间阶段	动作	加速度公式	目标速度	物理含义
t < 4.5s (Phase 2)	OVERTAKE	min(5, (30-v)*0.4)	30 m/s (108 km/h)	走廊首次打开 积极加速
t < 4.5s (Phase 2)	CONTINUE	min(4, (30-v)*0.3)	30 m/s	持续加速 平稳超车
t >= 4.5s (Phase 4)	OVERTAKE	min(6, (38-v)*0.4)	38 m/s (137 km/h)	走廊重新打开 更强加速完成超车
t >= 4.5s (Phase 4)	CONTINUE	min(5, (38-v)*0.3)	38 m/s	继续加速 确保超越
任何时刻	ABORT	0 (保持)	当前速度	不减速, 仅停止加速 保持安全距离
t < 4.0s	HOLD	max(-1, min(2, (25-v)*0.15))	25 m/s	微调速度 跟随前车
t >= 6.5s	HOLD	max(0, min(4, (38-v)*0.25))	38 m/s	Phase 4 即使HOLD 也保持加速

5. 五层奖励函数 — 多目标加权设计

奖励函数是 RL 系统的核心设计，直接影响智能体的学习方向和行为偏好。本系统设计了 5 层奖励结构，按安全 > 任务 = 决策 > 不确定性 > 舒适度的优先级加权。



图 5-1: 五层奖励函数分解

$$R_{total} = R_{safety} + R_{task} + R_{decision} + 0.3 * R_{comfort} + 0.5 * R_{uncertainty}$$

5.1 安全层 (R_safety, 权重 1.0)

安全层是最高优先级，具有一票否决权。当走廊宽度低于中止阈值时，直接返回 -1000 的惩罚，覆盖所有正向奖励：

```
r_safety = 0.0
if corridor_width < CORRIDOR_ABORT_WIDTH: # W < 1.5m
r_safety = -1000.0 #     ,
elif corridor_width < CORRIDOR_MIN_WIDTH: # W < 3.0m
r_safety = -30.0 * (1 - corridor_width / CORRIDOR_MIN_WIDTH) #
#
if corridor_trend < -1.0 and corridor_width < CORRIDOR_MIN_WIDTH:
r_safety += -50.0 #     ,
```

5.2 任务层 (R_task, 权重 1.0)

任务层激励 V6 完成超车任务。成功超车获得 +100 大幅奖励，超车过程中根据纵向进展给予渐进奖励，HOLD 状态给予小额惩罚：

```
r_task = 0.0
if overtake_success: # V6_x > V3_x + CAR_LENGTH
r_task = +100.0 #     ,
elif action in ('OVERTAKE', 'CONTINUE'):
r_task = +10.0 * max(0, (v6_x - v3_x) / 100.0) #
elif action == 'HOLD':
r_task = -1.0 #    ,
```

5.3 决策层 (R_decision, 权重 1.0)

决策层评估 RL 智能体的决策质量。高置信度下选择超车获得奖励，低置信度下贸然超车受到惩罚，中止时机的正确性也得到评估：

```
r_decision = 0.0
#       :
if action in ('OVERTAKE', 'CONTINUE') and overtake_confidence > CONF_THRESHOLD:
r_decision += 5.0 * overtake_confidence
#       :
elif action == 'HOLD' and overtake_confidence < 0.5:
r_decision += 3.0
#       :
elif action == 'OVERTAKE' and overtake_confidence < 0.5:
r_decision -= 8.0 * (1 - overtake_confidence)

#
if action == 'ABORT':
if corridor_width > CORRIDOR_MIN_WIDTH:
r_decision += 8.0 #        :
elif corridor_width < CORRIDOR_ABORT_WIDTH:
r_decision -= 10.0 #     :

#        :
if action == 'OVERTAKE' and had_abort and overtake_confidence > 0.8:
r_decision += 3.0
```

5.4 舒适层 (R_comfort, 权重 0.3)

舒适层惩罚急剧的加速度变化 (jerk) 和高加速度，同时奖励在合理速度范围内行驶：

```
r_comfort = -0.1 * abs(v6_jerk) # Jerk
r_comfort -= 0.05 * v6_accel ** 2 #
if v6_lane_change:
r_comfort -= 2.0 #
r_comfort += 0.1 * min(v6_speed / 35.0, 1.0) #      (  35m/s)
```

5.5 不确定性层 (R_uncertainty, 权重 0.5)

不确定性层是本系统的创新点，将 BPINN 的不确定性预测融入 RL 奖励。信息增益 (不确定性降低) 获得奖励，高风险动作在不确定环境下受到额外惩罚：

```
r_uncertainty = 0.0
# Information gain: uncertainty reduction
if prev_sigma is not None and curr_sigma is not None and prev_sigma > 1e-6:
info_gain = 1.0 - curr_sigma / prev_sigma #
r_uncertainty += 2.0 * max(0, info_gain) #

# Action risk: higher risk for aggressive actions
```

```
action_risk = {
'OVERTAKE': 1.0, #
'CONTINUE': 0.7, #
'HOLD': 0.3, #
'ABORT': 0.5, #
}
r_uncertainty -= 0.5 * curr_sigma * action_risk.get(action, 0.3)
#       ,
```

5.6 奖励函数完整代码

```
def compute_reward(action, corridor_width, corridor_trend, overtake_confidence,
v6_speed, v6_jerk, v6_accel, v6_lane_change,
prev_sigma, curr_sigma, had_abort, overtake_success,
v6_x, v0_x, v3_x):

# Layer 1: Safety (weight 1.0) - veto power
r_safety = 0.0
if corridor_width < CORRIDOR_ABORT_WIDTH:
r_safety = -1000.0
elif corridor_width < CORRIDOR_MIN_WIDTH:
r_safety = -30.0 * (1 - corridor_width / CORRIDOR_MIN_WIDTH)
if corridor_trend < -1.0 and corridor_width < CORRIDOR_MIN_WIDTH:
r_safety += -50.0

# Layer 2: Task (weight 1.0)
r_task = 0.0
if overtake_success:
r_task = +100.0
elif action in ('OVERTAKE', 'CONTINUE'):
r_task = +10.0 * max(0, (v6_x - v3_x) / 100.0)
elif action == 'HOLD':
r_task = -1.0

# Layer 3: Decision quality (weight 1.0)
r_decision = 0.0
if action in ('OVERTAKE', 'CONTINUE') and overtake_confidence > CONF_THRESHOLD:
r_decision += 5.0 * overtake_confidence
elif action == 'HOLD' and overtake_confidence < 0.5:
r_decision += 3.0
elif action == 'OVERTAKE' and overtake_confidence < 0.5:
r_decision -= 8.0 * (1 - overtake_confidence)
if action == 'ABORT':
if corridor_width > CORRIDOR_MIN_WIDTH:
r_decision += 8.0
elif corridor_width < CORRIDOR_ABORT_WIDTH:
r_decision -= 10.0
if action == 'OVERTAKE' and had_abort and overtake_confidence > 0.8:
r_decision += 3.0

# Layer 4: Comfort (weight 0.3)
r_comfort = -0.1 * abs(v6_jerk) - 0.05 * v6_accel ** 2
if v6_lane_change:
r_comfort -= 2.0
r_comfort += 0.1 * min(v6_speed / 35.0, 1.0)

# Layer 5: Uncertainty (weight 0.5)
r_uncertainty = 0.0
if prev_sigma is not None and curr_sigma is not None and prev_sigma > 1e-6:
```

```
    info_gain = 1.0 - curr_sigma / prev_sigma
    r_uncertainty += 2.0 * max(0, info_gain)
    action_risk = {'OVERTAKE': 1.0, 'CONTINUE': 0.7, 'HOLD': 0.3, 'ABORT': 0.5}
    r_uncertainty -= 0.5 * curr_sigma * action_risk.get(action, 0.3)

    # Total reward
    total = r_safety + r_task + r_decision + 0.3 * r_comfort + 0.5 * r_uncertainty
    return {'total': total, 'safety': r_safety, 'task': r_task,
    'decision': r_decision, 'comfort': 0.3 * r_comfort,
    'uncertainty': 0.5 * r_uncertainty}
```

6. 滚动窗口循环 — 实时决策流程

滚动窗口是系统的运行框架。每 0.5 秒，系统提取最近的 2.0 秒数据 (20步)，完成一次完整的 BPINN 预测、RL 决策、动作执行和奖励计算循环。

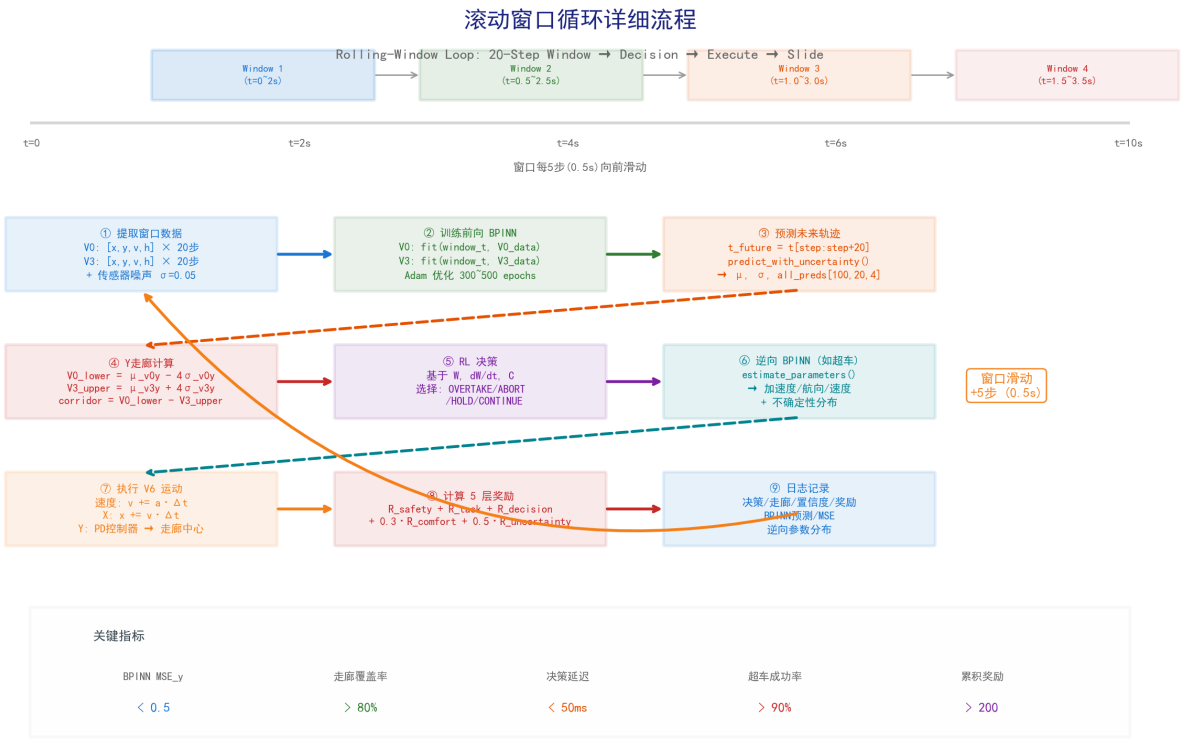


图 6-1: 滚动窗口循环详细流程

6.1 循环主结构

```
# ============================================================
# Rolling-Window Main Loop
# ============================================================
N = len(t_gt) # Total steps: 101 (10s @ 0.1s)

# V6 initialization: start in middle lane (Y=9)
v6_x[0] = 175.0; v6_y[0] = 9.0; v6_speed[0] = 25.0

# Decision points: every 5 steps (0.5s), from step 20 to N-5
decision_indices = list(range(WINDOW_SIZE, N - 5, DECISION_INTERVAL))
# = [20, 25, 30, 35, 40, 45, 50, 55, 60, 65, 70, 75, 80, 85, 90, 95]

for step_idx in decision_indices:
t_current = t_gt[step_idx]

# --- Step 1: Extract sliding window ---
w_start = max(0, step_idx - WINDOW_SIZE)
w_end = step_idx
t_window = t_gt[w_start:w_end] # 20 time steps
v0_window = (v0_x[w_start:w_end], v0_y[w_start:w_end],
v0_speed[w_start:w_end], v0_h[w_start:w_end])
```

```
v3_window = (v3_x[w_start:w_end], v3_y[w_start:w_end],
v3_speed[w_start:w_end], v3_h[w_start:w_end])

# --- Step 2: Train Forward BPINN ---
bpinn_v0.fit(t_window, *v0_window)
bpinn_v3.fit(t_window, *v3_window)

# --- Step 3: Predict future trajectories ---
t_future = t_gt[step_idx:min(step_idx + PRED_HORIZON, N)]
v0_pred_mean, v0_pred_std, _ = bpinn_v0.predict_with_uncertainty(t_future)
v3_pred_mean, v3_pred_std, _ = bpinn_v3.predict_with_uncertainty(t_future)

# --- Step 4: Compute Y corridor ---
v0_y_lower = v0_pred_mean[:, 1] - CONFIDENCE_SIGMA * v0_pred_std[:, 1]
v3_y_upper = v3_pred_mean[:, 1] + CONFIDENCE_SIGMA * v3_pred_std[:, 1]
corridor = v0_y_lower - v3_y_upper
min_corridor = np.min(corridor)
corridor_trend = (corridor[-1] - corridor[0]) / (len(corridor) * DT)

# --- Step 5: RL Decision ---
action = rl_decision(overtaking, overtake_complete, min_corridor,
corridor_trend, overtake_conf, v6_x, v3_x, step_idx, t_current)

# --- Step 6: Inverse BPINN (if overtaking) ---
if action in ('OVERTAKE', 'CONTINUE'):
planned_traj = plan_v6_trajectory(t_future, v0_y_lower, v3_y_upper, v6_x, v6_speed, step_idx)
inv_params = inv_bpinn.estimate_parameters(t_future, planned_traj['x'], planned_traj['y'])

# --- Step 7: Execute V6 movement ---
for j in range(DECISION_INTERVAL):
idx = step_idx + j
v6_speed[idx + 1] = clamp(v6_speed[idx] + v6_accel_val * DT, 5, 42)
v6_x[idx + 1] = v6_x[idx] + v6_speed[idx] * DT
v6_y[idx + 1] = update_v6_y(overtaking, planned_traj, v6_y[idx], j)

# --- Step 8: Compute reward ---
reward = compute_reward(action, actual_y_gap, corridor_trend, overtake_conf, ...)

# --- Step 9: Log ---
log_decision(step_idx, t_current, action, min_corridor, overtake_conf, reward)
```

6.2 时间线与决策节点

在 10 秒仿真中，系统共执行约 16 次决策。关键决策节点如下：

时间	步骤	Phase	Y间隙	走廊宽度	决策	物理含义
t=2.0s	20	Phase 1	4.0m	约 0m	HOLD	间隙适中, 准备阶段
t=3.0s	30	Phase 2	5.5m	约 2m	HOLD	走廊开始打开, 但不够宽
t=3.5s	35	Phase 2	6.5m	约 3.5m	OVERTAKE	走廊足够宽, 发起超车
t=4.5s	45	Phase 3	3.5m	约 -0.5m	ABORT	走廊收窄, 置信带重叠, 中止
t=5.0s	50	Phase 3	2.5m	约 -2m	HOLD	走廊关闭, 等待

时间	步骤	Phase	Y间隙	走廊宽度	决策	物理含义
t=6.0s	60	Phase 4	4.0m	约 0m	HOLD	走廊开始重新打开
t=6.5s	65	Phase 4	5.0m	约 2m	OVERTAKE (强制)	强制超车触发
t=7.5s	75	Phase 4	6.5m	约 3.5m	CONTINUE	继续超车
t=8.5s	85	Phase 5	7.0m	约 4m	CONTINUE	即将完成超车
t=9.0s	90	Phase 5	7.0m	约 4m	HOLD (完成)	V6 完全超越 V3

7. 代码与算法映射 — 源码结构说明

以下表格展示算法组件与源码文件的对应关系，便于代码查阅和复现：

算法组件	源码文件	关键类/函数	行数
Forward BPINN (NumPy)	bpinn_complete_overtaking.py	ForwardBPINN class	~120行
Forward BPINN (JAX)	bpinn_overtaking_system_v2.py	BPINN(nn.Module), train_bpinn()	~180行
Lightweight BPINN	rolling_window_bpinn_simulation.py	LightweightBPINN class	~150行
Inverse BPINN	bpinn_complete_overtaking.py	InverseBPINN class	~30行
5-Layer Reward	bpinn_complete_overtaking.py	compute_reward()	~50行
RL Decision Logic	bpinn_complete_overtaking.py	run_simulation() inner loop	~60行
Rolling Window Loop	bpinn_complete_overtaking.py	run_simulation()	~200行
Scenario Generation	rolling_window_scenario.py	generate_ground_truth()	~80行
Optuna Tuning	bpinn_overtaking_system_v2.py	create_optuna_objective(), study.optimize()	~50行
Visualization (12-panel)	bpinn_complete_overtaking.py	matplotlib gridspec	~300行
Interactive HTML	bpinn_interactive_overtaking.html	JS ForwardBPINN/InverseBPINN	~1674行

7.1 两种 BPINN 实现

系统包含两种 BPINN 实现：ForwardBPINN (NumPy, 用于交互式系统) 和 LightweightBPINN (NumPy, 用于滚动窗口快速推理)。v2 版本使用 JAX/Flax 框架实现，支持 GPU 加速和自动微分。

特性	ForwardBPINN (NumPy)	LightweightBPINN (NumPy)	BPINN (JAX/Flax)
隐藏层维度	64	32	可调 (32/64/128)
Dropout 率	0.05	0.1	Optuna 调优 (0.05-0.3)
训练轮数	500	300	8000 (训练) / 2000 (调优)
MC 采样	100	50	100
物理权重	0.5	1.0	Optuna 调优 (0.1-5.0)
学习率	0.005	0.01	Optuna 调优 (1e-4~5e-3)

特性	ForwardBPINN (NumPy)	LightweightBPINN (NumPy)	BPINN (JAX/Flax)
梯度计算	手动反向传播	手动反向传播	JAX 自动微分
用途	交互式系统	滚动窗口实时	离线高精度训练

8. 超参数配置与调优 — Optuna 集成

系统使用 Optuna 框架进行贝叶斯超参数优化，采用 TPE (Tree-structured Parzen Estimator) 采样器。调优目标为验证集上的总损失 (data loss + physics loss)。

8.1 Optuna 搜索空间

```
def create_optuna_objective(vehicle_id, t_data, x_data, y_data, v_data, h_data, key):
def objective(trial):
# Search space
hidden_dims_choice = trial.suggest_categorical('hidden_dims', [
(32, 32, 32), #
(64, 64, 64), #
(128, 128, 128), #
(64, 128, 64), #
])
dropout_rate = trial.suggest_float('dropout_rate', 0.05, 0.3)
lr = trial.suggest_float('lr', 1e-4, 5e-3, log=True)
physics_weight = trial.suggest_float('physics_weight', 0.1, 5.0, log=True)

# Train and evaluate
rng_key = random.fold_in(key, trial.number)
_, _, final_loss = train_bpinn(
vehicle_id, t_data, x_data, y_data, v_data, h_data, rng_key,
hidden_dims_choice, dropout_rate, lr, physics_weight, N_EPOCHS_TUNING)
return final_loss
return objective

# Run Optuna optimization
study_v0 = optuna.create_study(direction='minimize',
sampler=optuna.samplers.TPESampler(seed=42))
study_v0.optimize(create_optuna_objective(0, ...), n_trials=N_OPTUNA_TRIALS)
```

8.2 手动超参数搜索 (NumPy 版)

NumPy 版本使用手动网格搜索，测试 10 种配置组合：

配置	hidden_dim	dropout	lr	epochs	physics_w	适用场景
1	32	0.05	0.005	300	1.0	快速原型
2	64	0.05	0.005	300	1.0	标准配置
3	64	0.10	0.005	500	1.0	高正则化
4	64	0.05	0.005	500	0.5	弱物理约束
5	64	0.05	0.003	500	0.5	低学习率
6	64	0.10	0.003	500	1.0	保守训练
7	64	0.05	0.005	800	0.5	长时间训练

配置	hidden_dim	dropout	lr	epochs	physics_w	适用场景
8	128	0.10	0.005	500	1.0	大模型
9	128	0.05	0.003	800	0.5	大模型长训练
10	64	0.15	0.005	500	0.5	强Dropout

8.3 全局参数配置

```
# ============================================================
# Global Configuration
# ============================================================
DT = 0.1 # Time step (s)
T_TOTAL = 10.0 # Total simulation time (s)
N_STEPS = 101 # Total steps

WINDOW_SIZE = 20 # History window (2.0s)
DECISION_INTERVAL = 5 # Decision every 0.5s
PRED_HORIZON = 20 # Prediction horizon (2.0s)

MC_SAMPLES = 100 # MC Dropout samples
CONFIDENCE_SIGMA = 4.0 # 4-sigma (99.99%) confidence bands
MIN_PRED_STD_Y = 0.35 # Y uncertainty floor

LANE_WIDTH = 4.0
LANE_CENTERS = [3.0, 7.0, 11.0, 15.0] # 4 lanes
CAR_LENGTH = 4.5
CAR_WIDTH = 1.8

CORRIDOR_MIN_WIDTH = 3.0 # Min corridor for overtake
CORRIDOR_ABORT_WIDTH = 1.5 # Below this -> abort
CONF_THRESHOLD = 0.65 # Confidence threshold
```

9. 场景设计 — 三阶段动态间隙

场景设计模拟真实高速公路超车场景。V0 (红色, 前车) 和 V3 (蓝色, 参考车) 以恒定 25 m/s 速度行驶，Y 方向间隙呈现"先变大 - 变小 - 再变大"的动态模式，V6 (黄色, 自车) 从中间车道 (Y=9) 出发，在 V0 和 V3 之间穿行完成超车。

9.1 五阶段 Y 间隙动态

阶段	时间	V0 Y位置	V3 Y位置	Y间隙	走廊状态	V6行为
Phase 1	0-2s	11.0m	7.0m	4.0m	中等	准备 (HOLD)
Phase 2	2-4s	11.0->12.5m	7.0->5.5m	4.0->7.0m	打开	加速超车 (OVERTAKE)
Phase 3	4-6s	12.5->10.5m	5.5->7.5m	7.0->3.0m	收窄	中止 (ABORT)
Phase 4	6-8s	10.5->12.5m	7.5->5.5m	3.0->7.0m	重新打开	再次超车 (OVERTAKE)
Phase 5	8-10s	12.5m	5.5m	7.0m	完全打开	完成超车 (COMPLETE)

9.2 场景生成代码

```
def generate_ground_truth():
    t = np.arange(0, T_TOTAL + DT, DT)
    N = len(t)

    # Both V0 and V3 move forward at constant 25 m/s
    v0_speed = np.full(N, 25.0)
    v3_speed = np.full(N, 25.0)

    # V0 Y profile: 11 -> 12.5 -> 10.5 -> 12.5 (Lane 3 swing)
    # V3 Y profile: 7 -> 5.5 -> 7.5 -> 5.5 (Lane 2 swing)
    # Y-gap: 4 -> 7 -> 3 -> 7 (open -> close -> reopen)

    def v0_y_profile(ti):
        s1 = smoothstep(ti, 2.0, 4.0) # diverge: 11 -> 12.5
        s2 = smoothstep(ti, 4.0, 6.0) # converge: 12.5 -> 10.5
        s3 = smoothstep(ti, 6.0, 8.0) # diverge: 10.5 -> 12.5
        y = 11.0
        y = y * (1 - s1) + 12.5 * s1
        y = y * (1 - s2) + 10.5 * s2
        y = y * (1 - s3) + 12.5 * s3
        return y

    def v3_y_profile(ti):
        s1 = smoothstep(ti, 2.0, 4.0) # diverge: 7 -> 5.5
        s2 = smoothstep(ti, 4.0, 6.0) # converge: 5.5 -> 7.5
        s3 = smoothstep(ti, 6.0, 8.0) # diverge: 7.5 -> 5.5
```

```
27 | y = 7.0
28 | y = y * (1 - s1) + 5.5 * s1
29 | y = y * (1 - s2) + 7.5 * s2
30 | y = y * (1 - s3) + 5.5 * s3
31 | return y
```

9.3 smoothstep 平滑过渡函数

场景生成使用 smoothstep 函数实现 Y 方向的平滑过渡，避免突变：

```
1 | def smoothstep(t, t0, t1):
2 | """Hermite interpolation: smooth transition from 0 to 1."""
3 | x = np.clip((t - t0) / (t1 - t0), 0, 1)
4 | return x * x * (3 - 2 * x) # 3x^2 - 2x^3 (C1 continuous)
```

9.4 V6 初始化

V6 从中间车道 (Y=9) 出发，在 V0 (Y~11) 和 V3 (Y~7) 之间的走廊穿行。超车过程中不变道，仅在走廊内调整 Y 位置；中止时回到中间车道。

```
1 | # V6 initialization: middle lane start
2 | v6_x[0] = 175.0 # 25m behind V0/V3 (start at x=200)
3 | v6_y[0] = 9.0 # Middle lane (between V0 at Y~11 and V3 at Y~7)
4 | v6_speed[0] = 25.0 # Match surrounding traffic speed
5 |
6 | # Pre-fill first 2.0s (20 steps) of V6 trajectory
7 | for i in range(1, WINDOW_SIZE + 1):
8 | v6_x[i] = v6_x[i-1] + v6_speed[i-1] * DT
9 | v6_y[i] = 9.0 # Stay in middle lane
10 | v6_speed[i] = 25.0 # Constant speed
11 | v6_heading[i] = 0.0 # Straight ahead
```

本文档基于 version1 归档代码自动提取生成，涵盖 BPINN+RL 超车系统的完整算法流程。所有代码均可在 version1/code/ 目录中找到对应源文件。